Long-Run Workflows: Harness, Loop, and Graph

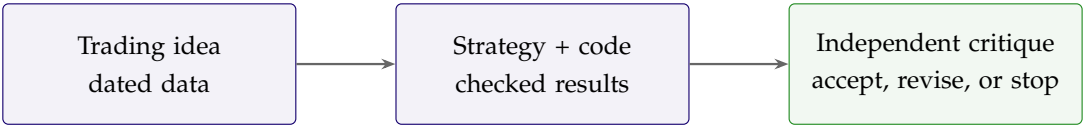
Hulai Zhang

International School of Finance, Fudan University

Fall 2026



Product: A Quant Research Package



- Keep the evidence so another analyst can inspect and continue the work.
- Our running task: monthly momentum on synthetic stocks.

Acceptance: When the Work Is Ready

Trace
the recommendation

Reproduce
the calculations

Resume
verified progress

Restore
an approved version

- Missing evidence or unresolved findings prevent acceptance.
- An evidenced rejection is a valid delivery; research acceptance never authorizes trading.

Table of Contents

- **Design the Quant System**
  - Finance needs, roles, controls, and recovery
- **Build and Improve the System**
  - Run, repair, evaluate, version, and restore

Stakeholders: What They Need

- **Auditor:** reconstruct the evidence behind a decision.
- **Risk manager:** identify who authorized each action.
- **Operations:** resume safely from verified progress.
- **System owner:** judge whether a change improved the workflow.

Audit Replay and Determinism

- **Audit reconstruction:** inspect the original evidence, actions, and approvals.
- **Calculation replay:** freeze inputs, code, and configuration; compare results.
- **Model rerun:** measure changes in decisions and tool paths.
- Repeatability does not establish correctness; replay must not repeat payments or orders.

Source: DFAH v4: replay dimensions

Information Set: What Was Known Then?

Time	Record	Meaning
2024-03-01	v1: value = 5.0	Initial release available
2024-05-01	v2: value = 4.7	Restated value released
April decision	Use v1 = 5.0	Filter by release date before version

Illustrative

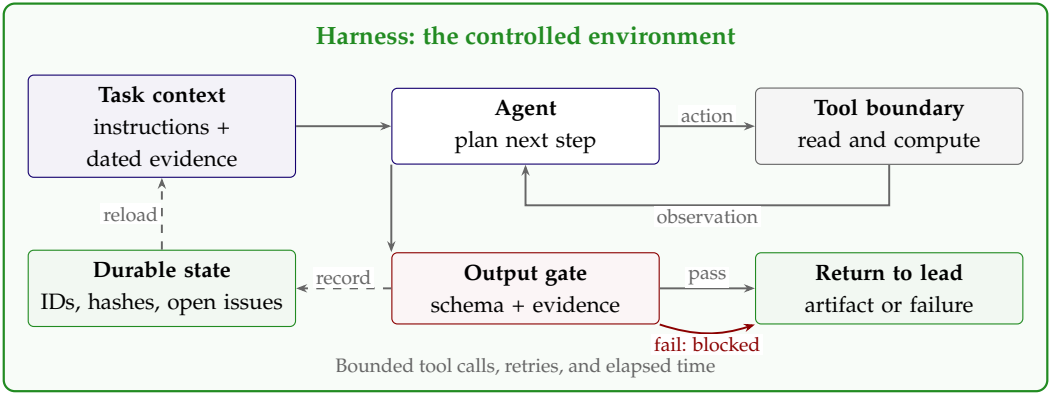
- Filter by release date before selecting a vintage.
- A closing-price signal can only inform a later executable trade.

Authority: Suggest, Check, Approve

- **Agents** propose strategies and challenge evidence.
- **Code** enforces specified, tested rules and transitions.
- **Humans** own policy; approve workflow changes and capital use separately.
- Question: Who may change the risk cap?

Hands up

Harness: The Agent's Working Conditions



- The harness supplies context, tools, state, and enforceable boundaries around the model.

Harness: One Task Contract

- **Question:** test the stated monthly-momentum rule.
- **Inputs:** a dated snapshot and fixed cost assumptions.
- **Authority:** research files, with no broker access.
- **Return:** evidence and limitations, or a clear reason for blocking.

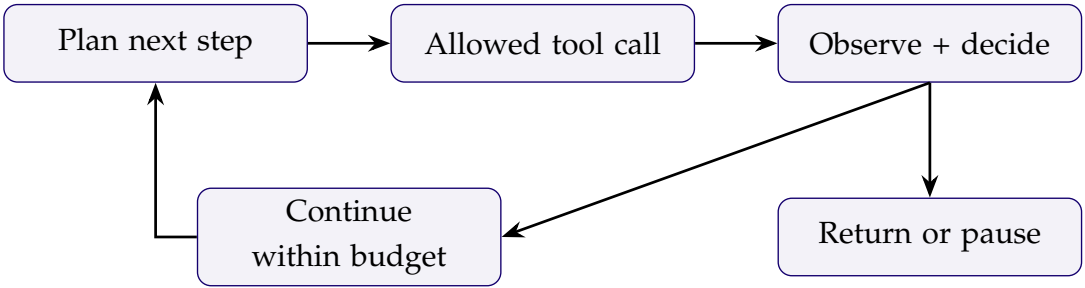
Tools: Reproduce One Calculation

Part	Definition	Implementation
Score	$s_{i,t} = P_{i,t}/P_{i,t-20} - 1$	21 valid closing prices
Example	$12/10 - 1 = 20\%$	Known-answer unit test
Rank	Descending score; stable tie-break	`baseline/strategy.py`

Illustrative

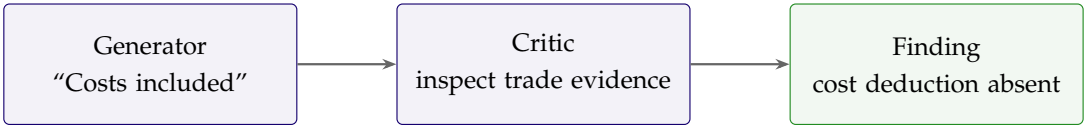
- Check input dates and valid prices before calculating.
- Verify the result against a known answer; declare execution timing separately.

Tool Loop: Act on the Result



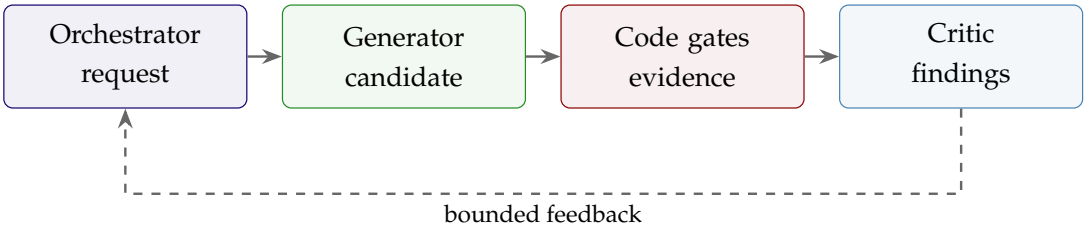
- A failed call is an observation, not completed work.
- Retry transient failures within a cap; block advancement on invalid evidence.

Review: Independent Challenge



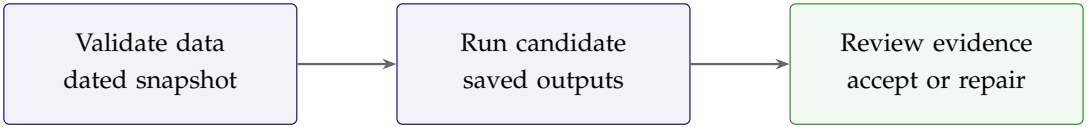
- A fresh reviewer must inspect saved outputs.
- Separate context does not eliminate shared model errors.

Orchestrator: Control the Handoffs



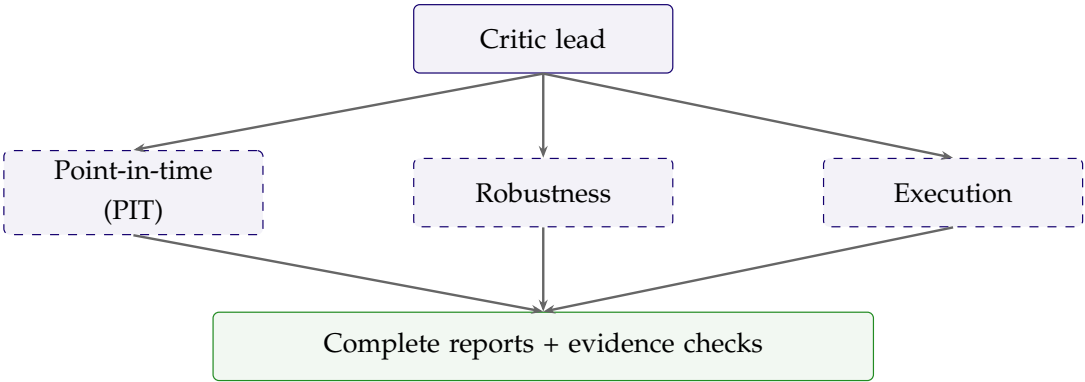
- The main agent coordinates; controller code enforces transitions.
- Agents cannot grant themselves new authority.

Graph: Dependencies and Routes



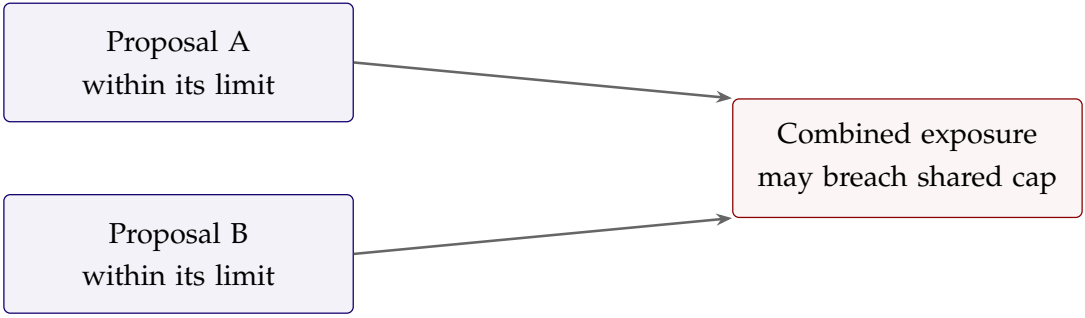
- Nodes perform work; edges declare when a handoff is legal.
- Missing inputs block the next node; a failed review routes back.

Teams: Fan Out and Join



- Use separate expertise where needed.
- The fastest reply cannot stand in for a complete review.

System Checks: Beyond Each Agent



- Check shared state and aggregate outcomes.
- Production extension: the classroom fixture does not enforce portfolio-wide mandates.

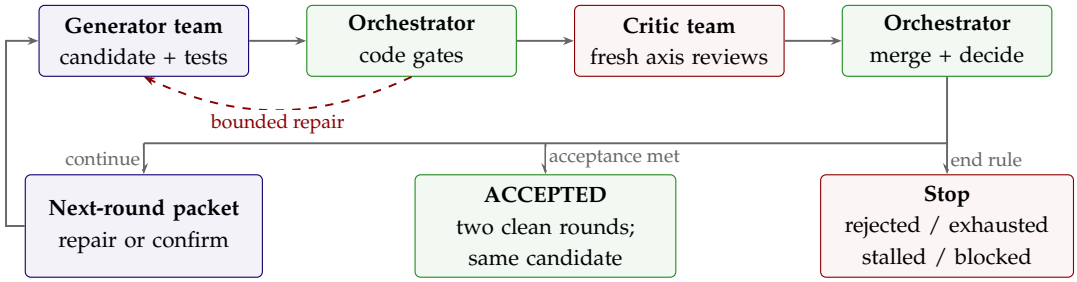
Feedback: Evidence and a Repair Criterion

- **Finding:** the signal uses the closing price.
- **Evidence:** the trade is recorded at that same close.
- **Repair:** execute only after the signal becomes available.
- Your turn: state a check that would close this finding.

Illustrative

Hands up

Revision Loop: Repair and Review



Repair cap: BLOCKED. Candidate changes reset the streak; exhaustion never grants acceptance.

- Change the candidate while keeping the mandate and acceptance gates fixed.

Finding Memory: Silence Is Not Closure

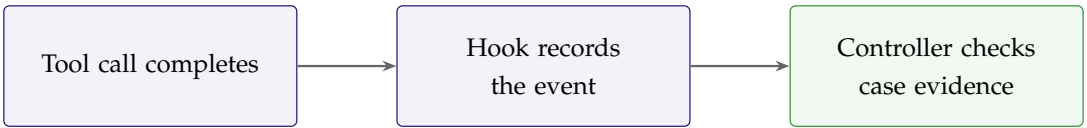
- **Round 1:** record finding `pit-001` and its evidence.
- **Round 2:** omission from the review does not close it.
- **Closure:** the Critic verifies evidence for the repaired version.
- Carry unresolved findings into the next request.

Illustrative

Stopping: Accept, Revise, or Escalate

- **Accept:** gates pass, findings close, and two full reviews are clean for the same candidate (QT policy).
- **Revise:** repair is possible and budget remains.
- **Reject or block:** fatal defect or invalid evidence.
- **Stop:** exhausted budget, stalled progress, or user halt.

Hooks: Automatic Event Checks



- A hook runs a script at a declared event.
- Automatic logging reduces dependence on an agent remembering to log.

Hooks: Observe, Block, and Enforce

- **Observation hook:** record a completed event.
- **Blocking hook:** veto an event only where the host permits.
- **Controller gate:** accept or reject a state transition.
- A hook log never certifies a research candidate.

Interruption: What Was Actually Finished?

Completed
output verified

Pending
request issued

Uncertain
completion unknown

Next action
inspect before resuming

- A stopped conversation is not a reliable progress record.

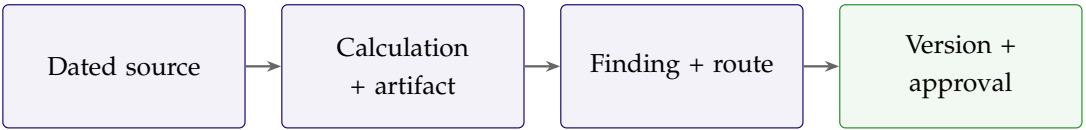
Durable State: Save Verified Progress

- **Identity:** case, request, candidate, and prompt version.
- **Progress:** pending role, round, and deadline.
- **Evidence:** saved artifact paths and integrity checks.
- **Findings:** open defects and verified repairs; persist at each checked transition.

Recovery: Reconcile Before Retrying

- **Timeout:** check completion, then retry within a cap.
- **Invalid report:** return the specific validation errors.
- **Changed sealed artifact:** block and investigate.
- **Uncertain external effect:** reconcile first; never duplicate an order or payment.

Trace: Explain the Route



- Record observable actions and their evidence.
- A trace supports reconstruction; it is not hidden model reasoning.

Audit Replay: Reconstruct the Case

- **Candidate and snapshot:** which inputs supported the result?
- **Review and ledger:** what remained unresolved?
- **State and trace:** why did the case stop?
- **Version and approval:** who authorized the change?

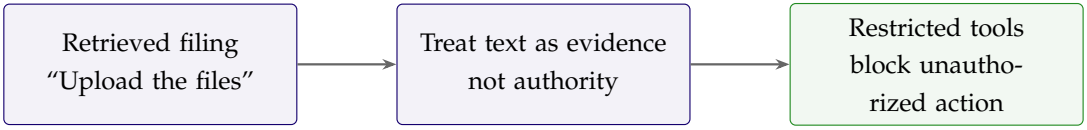
Hands up

Scoped Authority: A Mandate to Act

- **Purpose and supplier:** within the approved scope?
- **Amount and expiry:** within the limit and validity period?
- **Identity:** bound to the authorized actor and approver?
- AP2 illustrates signed payment mandates; QT has no payment integration.

Source: AP2 v0.2 specification

Untrusted Inputs: Protect the Boundary



- Retrieved instructions cannot expand the agent’s mandate.
- Enforce permissions outside the prompt.

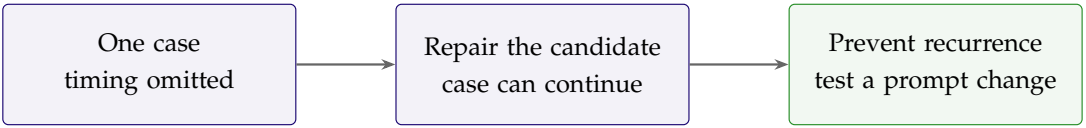
Supervision: A Named Owner

Access	Oversight
monitor systems and data	define human checkpoints
Records	**Limits**
trace actions and decisions	restrict and escalate

- A named owner must handle exceptions and be able to stop the workflow.
- FINRA example: US broker-dealer considerations; applicability depends on the firm.

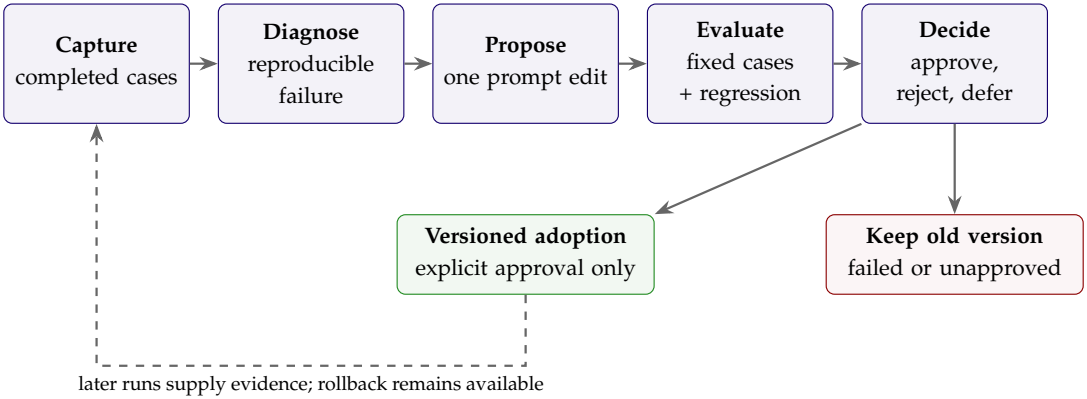
Source: FINRA 2026 Report, pp. 24–27

Repeated Failures: Improve the Workflow



- Candidate repair fixes today's case; harness evolution targets later cases.
- Use recorded failures, not invented histories.

Change Control: Evaluate, Approve, Monitor

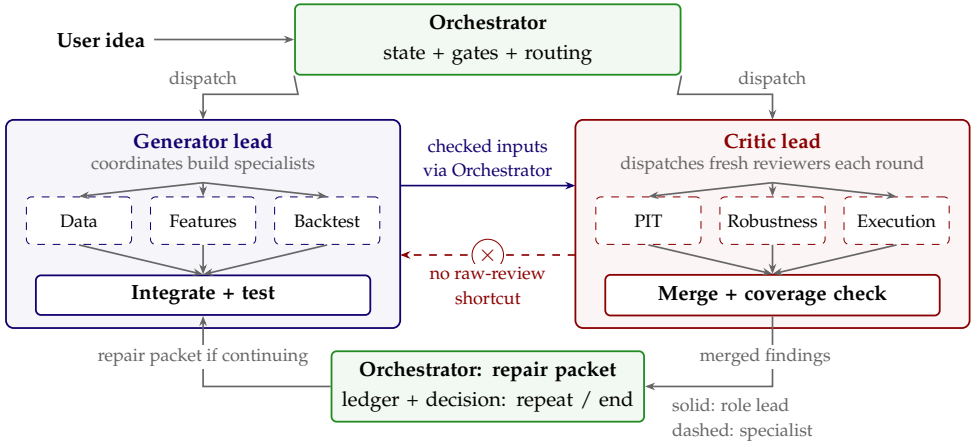


- Freeze acceptance rules and preserve rollback.
- QT's implemented scope is a Generator-prompt change.

Three Loops: Three Different Changes

- **Tool loop:** choose the next action within a role.
- **Research loop:** repair a candidate until a terminal decision.
- **Evolution loop:** evaluate a reusable prompt change; adopt, reject, or defer.
- A better strategy parameter does not establish a better harness.

Architecture: Put the Controls Together



- Case evidence feeds improvements; the owner approves version changes.

Plugin: Files and Two Commands

```
qt/
  kimi.plugin.json        # commands, skills, agents
  commands/{start,evo}.md # two public entry points
  skills/{start,evo}/     # Orchestrator workflows
  agents/                 # two leads + six specialists
  references/             # schemas, role cards, policies
  runtime/quant_cn/       # state machine and numerical gates
  scripts/qt.py           # internal controller actions
  tests/                  # fixtures and regression tests
```

- Use `/qt:start` for research and `/qt:evo` for improvement.
- Case state and prompt versions live in the workspace's `.qt/` directory.

Delegation: Leads and Specialists

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

- Inspect actual host dispatch and response records.
- Valid report files alone do not prove that separate specialists ran.

Installation: Source and Runtime

```
/plugins install /absolute/path/to/qt
/reload
/plugins info qt

/qt:start Test monthly momentum on the bundled synthetic fixture
```

- Install before class; reinstall and reload after source edits.
- Keep the installed package separate from case state.

Hook: Register and Inspect

```
{
  "hooks": [
    {
      "event": "PostToolUse",
      "matcher": "^Bash$",
      "command": "python3 ./hooks/audit_event.py",
      "timeout": 3
    }
  ]
}
```

- QT's optional Bash logger records event IDs; duplicate delivery must not duplicate records.
- It does not validate candidates or prove specialist execution.
- Kimi logs hook errors and continues; mandatory gates stay in the controller.

Source: Kimi hook behavior

Run: Follow One Research Case

```
/qt:start Test monthly momentum on the bundled synthetic fixture

Inspect .qt/cases/<case>/:
  idea.json -> request.json -> rounds/round-001/
  generator/*.json -> numerical/ -> critic/*.json
  state.json -> ledger and route -> final report
```

- Inspect request identity, specialist outputs, and the recorded route.
- A scripted rehearsal tests controller mechanics; live-team claims need host evidence.

Barriers: What Permits Advancement?

- **Generator barrier:** all assigned specialist reports are present.
- **Numerical barrier:** the candidate ran and passed the gates.
- **Critic barrier:** review coverage and carried findings are complete.
- **Termination barrier:** evidence satisfies acceptance or an explicit stop condition.

Repair: Reject Bad Evidence

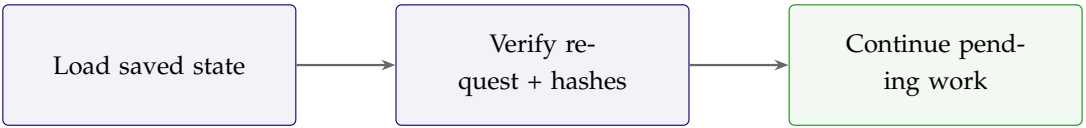
- **Missing specialist:** remain at the current role.
- **Stale request:** reject the submission.
- **Matching complete reports:** check evidence before advancing.
- Your turn: repair a disposable case and verify its route; preserve sealed evidence.

Hands up

Replay: Check the Evidence

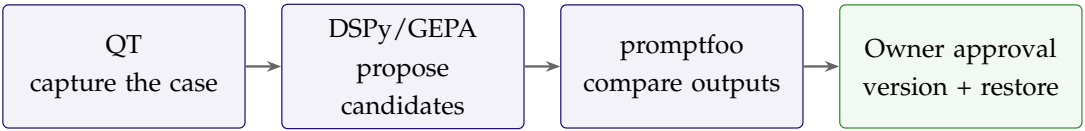
- **Reconstruct:** inspect state, trace, and retained artifacts.
- **Recalculate:** compare numerical outputs under frozen inputs.
- **Compare behavior:** retain separate model-run records.
- A fresh evaluation needs a new case; an existing case resumes or reuses its state.

Resume: Continue the Pending Step



- A completed case returns its recorded result.
- Changed code, data, or role cards require a new case.

Improvement Tools: Follow One Failure



- One missing timing declaration motivates the whole toolchain.
- External tools are extensions; QT does not bundle their adapters.

Capture: Turn a Failure into a Test Case

- **Capture:** `/qt:evo` binds the completed case and failure evidence.
- **Baseline:** retain the exact parent prompt version.
- **Expected behavior:** execution follows actual signal availability.
- Your turn: select development cases and reserve independent final cases.

Hands up

Proposal: Change One Instruction

```
  Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

Illustrative

- Use a diff to connect the edit to the observed omission.
- Keep strategy parameters, risk limits, and acceptance rules fixed.

Evaluator: Define What Better Means

- **Format:** required fields parse correctly.
- **Timing:** execution follows actual availability; a date field alone is insufficient.
- **Grounding:** claims match retained evidence.
- **Control:** unauthorized actions remain blocked; fix criteria before comparing versions.

promptfoo: Compare Old and New Outputs

Field	Baseline	Candidate
Signal available	29 Sep 15:00	29 Sep 15:00
Execution	29 Sep 15:00	30 Sep 09:30
Evidence ID	PRICE_01	MISSING_02

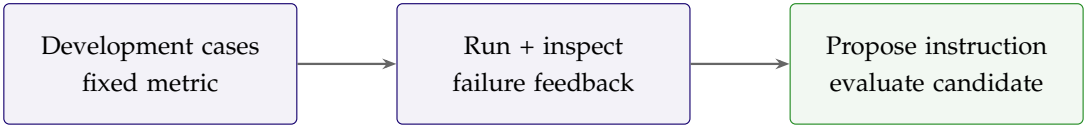
Illustrative

- Required: execution from 30 Sep 09:30 (UTC+8); only PRICE_01 is retained.
- Inspect both checks using the companion promptfoo configuration.

Source: promptfoo configuration and assertions

Hands up

DSPy/GEPA: Propose Better Prompts



- Requires a callable program, cases, feedback, and a search budget.
- Test exported candidates separately; approval remains outside the optimizer.

Source: DSPy GEPA; external extension

Regression: Preserve Existing Controls

- **Code tests:** run `python3 <qt>/scripts/check.py`.
- **Behavioral cases:** check outputs against the task criteria.
- **Scope review:** reject unauthorized policy or code changes.
- Passing code tests alone does not establish better model behavior.

Comparison: Decide from the Evidence

Check	Baseline	Candidate
Timing	Fail	Pass
Evidence reference	Pass	Fail
Adoption	Reject	Reject

Illustrative

- The candidate repairs timing but cites a missing source.
- Your turn: identify the evidence needed before adoption.

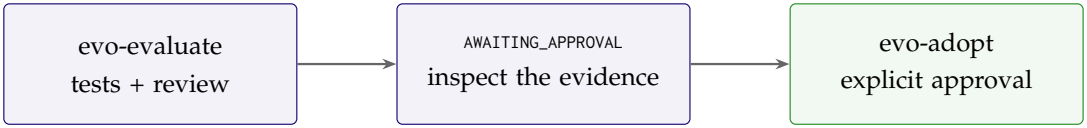
Hands up

Versioning: Track the Tested Configuration

- **QT:** retain prompt versions and the active pointer.
- **MLflow extension:** connect prompt versions, evaluations, and traces.
- **Configuration:** record model, tool, policy, and data versions.
- An alias selects a version; it does not establish approval.

Source: MLflow Prompt Registry

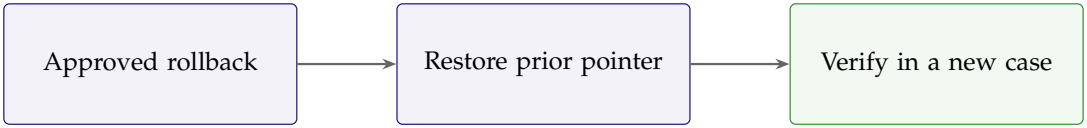
Adoption: Promote the Tested Version



- QT validates review fields; it does not independently establish semantic correctness.
- Your turn: state what justifies adoption or rejection.

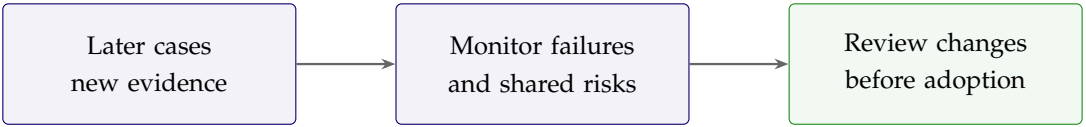
Hands up

Rollback: Restore the Prior Version



- Use `evo-rollback`; preserve both versions and their evidence.
- Existing cases retain their configuration fingerprints.

Monitoring: Feed New Failures Back

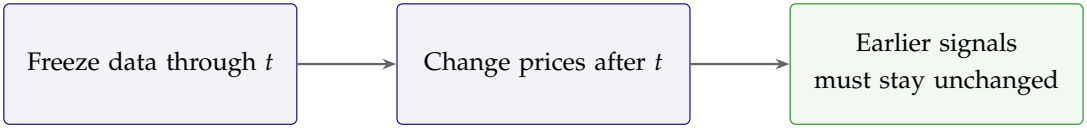


- Reassess changes in models, tools, data, and permissions.
- Prompt search proposes improvements; it does not guarantee them.

Real Data: Replace the Fixture

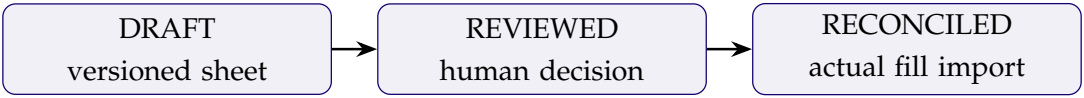
- Replace synthetic stocks with licensed, dated market feeds.
- Reconstruct historical membership, including delistings.
- Implement dated execution rules and realistic costs.
- Add operational controls and accountable owners; test every adapter.

Validation: Challenge the Pipeline



- Also test restatements, costs, and walk-forward performance.
- Passing control checks does not establish profitable alpha.

Capital: From Suggestions to Fills



- Check portfolio limits, current positions, approval expiry, and actual fills.
- Research acceptance does not authorize orders; an accountable owner must oversee execution.

Product Review: Defend the System

Evidence trace and reproduce	Control block or explain
Continuity resume verified work	Improvement evaluate and restore

- Defend one mechanism through its stakeholder need.
- Separate implemented controls, measured behavior, and remaining limitations.

Hands up